Java y programación orientada a objetos

Guía de fundamentos, métodos, clases, colecciones y excepciones

CódigoClaro · Material didáctico de apoyo · Edición octubre de 2026

Cómo estudiar este material

Lee el concepto, escribe los ejemplos sin copiar y modifica los casos de prueba. Los fragmentos son ejemplos didácticos: adapta nombres, tipos y validaciones al enunciado real.

1. Fundamentos de Java

Java es un lenguaje tipado: cada variable tiene un tipo. El código se organiza en clases y métodos. El método main es el punto de entrada habitual de una aplicación de consola.

```
public class Main {
    public static void main(String[] args) {
        String nombre = "Ana";
        int edad = 20;
        System.out.println(nombre + " tiene " + edad);
    }
}
```

Tipos y conversiones

Tipos primitivos: byte, short, int, long, float, double, char y boolean. String es una clase, no un primitivo. Evita conversiones que pierdan información y valida los datos introducidos.

```
int cantidad = 3;
double precio = 4.50;
double total = cantidad * precio;
boolean disponible = total > 0;
```

2. Control de flujo

```
if (nota >= 5) {
    System.out.println("Aprobado");
} else {
    System.out.println("Recuperación");
}

for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Usa if/else para decisiones, switch para alternativas discretas, for cuando conoces el recorrido y while cuando depende de una condición. Evita bucles cuya condición nunca cambia.

3. Métodos y arrays

```
static int sumar(int a, int b) {
    return a + b;
}

int[] notas = {7, 8, 6};
int suma = 0;
for (int nota : notas) suma += nota;
double media = (double) suma / notas.length;
```

Un método debe tener una responsabilidad clara. Los arrays tienen tamaño fijo y los índices empiezan en cero. Comprueba límites antes de acceder a una posición.

4. Programación orientada a objetos

Una clase define estado y comportamiento; un objeto es una instancia. Encapsula los atributos, utiliza constructores para inicializar objetos y expón métodos que mantengan las invariantes.

```
public class Cuenta {
    private double saldo;
    public Cuenta(double saldoInicial) {
        if (saldoInicial < 0) throw new IllegalArgumentException();
        saldo = saldoInicial;
    }
    public void ingresar(double cantidad) {
        if (cantidad > 0) saldo += cantidad;
    }
    public double getSaldo() { return saldo; }
}
```

Herencia, interfaces y polimorfismo

Herencia expresa una relación “es-un”; una interfaz expresa un contrato. Programa contra interfaces cuando sea útil y favorece la composición si evita jerarquías rígidas. Sobrescribe métodos con @Override.

5. Colecciones

```
List<String> nombres = new ArrayList<>();
nombres.add("Luz");
Set<String> unicos = new HashSet<>(nombres);
Map<String, Integer> puntos = new HashMap<>();
puntos.put("Luz", 10);
```

List conserva orden y admite duplicados; Set evita duplicados; Map relaciona claves con valores. Para comparar objetos por contenido, implementa correctamente equals y hashCode. Para ordenar, usa Comparator o Comparable.

6. Excepciones y ficheros

```
try {
    int n = Integer.parseInt("42");
} catch (NumberFormatException e) {
    System.out.println("Número no válido");
}
```

Captura excepciones que puedas manejar. No uses catch vacío. Para ficheros, prefiere try-with-resources para cerrar recursos automáticamente.

Ejercicios de autoevaluación

• Crea un método que reciba un array de notas y devuelva la media.

- Diseña una clase Producto con nombre, precio y un método para calcular precio con IVA.
- Cuenta cuántas veces aparece cada palabra en una lista usando Map.
- Explica por qué == y equals no significan lo mismo para String.

Checklist de examen

- ¿Puedes explicar encapsulación, herencia, abstracción y polimorfismo con ejemplos?
- ¿Controlas ArrayList, LinkedList, HashSet, TreeSet y HashMap?
- ¿Sabes validar entrada, controlar errores y dividir un programa en métodos?